# Static Type Checking for Forth by Symbolic Stack Effects

Jaanus Pöial
Tallinn University of Technology

**Abstract**

This paper describes a source-level static checker intended for Forth systems whose vocabulary and parsing rules can be described externally. The checker reads a type hierarchy, stack-effect and parser specifications, and Forth source. It represents each phrase by a pair of symbolic stacks. Indexed symbols retain the identity of cells moved or copied by words such as `DUP`, `OVER`, and `SWAP`; subtype-aware unification checks adjacent effects and propagates the most precise available type. Alternative paths are reduced to a common stack contract. Repetition uses a finite one-execution versus two-execution test rather than a general loop invariant calculation. Parser words, defining words, literals, and control structures are profile data, so checking is not tied to one fixed Forth syntax. The account concentrates on the choices that matter to a Forth implementor, gives a complete worked example, and states the important limits of the current native Gforth implementation.

**Keywords:** Forth; stack effect; static checking; symbolic execution; subtyping.

## 1 The problem in Forth terms

Forth programmers already use stack comments as interfaces:

```
: SQUARED  ( n -- n^2 )  DUP * ;
```

A useful checker must do more than count the cells on either side of `-`. It must know that both inputs of `*` are numeric, that `DUP` produces two references to the same abstract value, and that a word requiring a character address may accept an aligned address. It must also follow the outer interpreter: `:` consumes a name and starts a definition, `;` ends one, comments consume source text, and immediate control words combine non-linear source regions.

The evaluator in this project performs that work without running the program. It takes three independent inputs:

1. a finite set of types, aliases, and subtype declarations;
2. a dictionary of word effects plus parser and control metadata;
3. the source to be checked.

The same engine can therefore check a small teaching vocabulary, the bundled Forth-2012-like profile, or a system-specific profile. The executable is `gforth-evaluator.fs`; a Java version consumes the same files.

This is deliberately a checker for declared Forth interfaces, not a claim to infer the behavior of arbitrary native code. Trust begins at the primitive specifications. If `@` is declared incorrectly, no analysis of its clients can repair the error. Within that boundary, the checker infers colon definitions, checks linear compositions, and gives source-oriented clash diagnostics.

## 2 Effects with cell identity

### 2.1 The visible suffix and its frame

An effect is written in normal Forth order, with top of stack at the right:

$$(L\ --\ R).$$

It describes only the suffix touched by the phrase. An arbitrary deeper frame is preserved. Thus `( n - n )` is applicable to `F n`, not only to a one-cell data stack. This implicit frame is also why effects with different displayed depths can compose or describe two branches of one control structure.

The empty effect `( - )` is identity. A clash is a separate failure value; it is not an effect and absorbs subsequent composition. This keeps a reported error from being mistaken for a word that happens to have no stack behavior.

### 2.2 Types and indexed wildcards

A profile declares canonical types and aliases. A small address and numeric hierarchy may include

$$\texttt{a-addr} < \texttt{c-addr} < \texttt{addr} < \texttt{x}, \qquad \texttt{char} < \texttt{+n} < \texttt{n} < \texttt{x}.$$

Here "<" means "more specific than." The loader resolves aliases and computes transitive relationships. When two boundary cells are compared, the checker keeps the more specific type if the types are comparable; unrelated types clash. It does not search the hierarchy for a third, common subtype. That simple rule makes both behavior and diagnostics predictable for profile authors.

A type name may carry an index:

```
DUP   ( x[1] -- x[1] x[1] )
SWAP  ( x[2] x[1] -- x[1] x[2] )
OVER  ( x[2] x[1] -- x[2] x[1] x[2] )
DROP  ( x -- )
```

Equal positive indices mean equal symbolic cells. They express the dataflow that an ordinary unindexed effect omits. An unindexed `x` is fresh on each use; two plain occurrences do not assert equality. Indices are logical variables, not addresses and not runtime tags. Every fetched dictionary effect is cloned and its indices are freshened before use, so two calls of `DUP` cannot accidentally share a variable.

# 3 How a Forth phrase is composed

Suppose the current phrase has effect $(L_1 - - R_1)$ and the next word has $(L_2 - - R_2)$. Composition compares $R_1$ and $L_2$ from the top of stack inwards. For each pair it performs the following steps:

1. check that their types are comparable and select the more specific;
2. create one fresh symbol of that type;
3. replace both old symbols everywhere in both working effects;
4. remove the matched output/input pair and continue.

Replacing *every* occurrence is important. A numeric constraint found at one boundary must reach all copies made by `DUP`, and a constraint on one side of `SWAP` must follow the cell to its new position. When one touching side is exhausted, unmatched requirements or results are retained, with the untouched stack frame inserted in the appropriate place.

For example, use these declarations:

```
literal integer  ( -- n )
+                ( n n -- n )
DUP              ( x[1] -- x[1] x[1] )
```

The phrase `1 2 + DUP` reduces as follows:

$$(- - n) \cdot (- - n) = (- - n\ n),$$

$$(- - n\ n) \cdot (n\ n - - n) = (- - n),$$

$$(- - n) \cdot (x[1] - - x[1]\ x[1]) = (- - n[1]\ n[1]).$$

The final match specializes `x` to `n`; global substitution applies that choice to both outputs. The native trace makes the same process visible:

```
1    ( -- n )
2    ( -- n )
+    ( n n -- n[1] )
DUP  ( n[1] -- n[1] n[1] )
< n[1] n[1]
```

This procedure is deterministic. Fresh-index numbers may differ between runs or implementations, but consistent renaming does not alter the effect. The older algebraic presentation relates untyped stack effects to a polycyclic monoid [2, 3]; for using the checker, boundary matching and whole-effect substitution are the essential facts.

## 3.1 Compaction and a precision caveat

Working effects are normalized after a successful operation. Wildcard numbers are made small and deterministic. An index explicitly written by the profile author remains visible. An implicit identity that has too few occurrences may be printed without its index; for example an internally created `( n[7] - n[7] )` can become `( n - n )`.

Compaction produces readable inferred comments, but the implementation also stores the compacted value. It can therefore forget a pairwise equality that would help a later word. In analysis terminology this is a widening: stack height and the type at each position remain valid, while correlation precision can decrease. A future version should preserve raw identities internally and compact only when printing.

# 4 Branches and control words

Two arms of an `IF` must be represented by one contract for the code that follows. The checker calls the combining operation `glb`. The operation is variance-aware: aligned inputs retain the more specific comparable type required by either arm, whereas aligned outputs retain the more general comparable type guaranteed after either arm. An output identity is retained only when both arms establish the same pair of source identities.

A mismatch in visible depth is accepted when the effect with $k$ extra inputs also has exactly $k$ extra outputs. The shorter effect is then understood to preserve an implicit $k$-cell deep prefix, and its active suffix is aligned with the longer effect. Otherwise the merge clashes. The implementation still requires aligned types to be directly comparable; it does not search for a third common type.

Consider two store-like branch effects requiring respectively

$$(\texttt{x a-addr} -- --), \qquad (\texttt{x c-addr} -- --).$$

Because `a-addr` is more specific than `c-addr`, their common contract requires `a-addr`. Conversely, output types are widened: the branches `@ ( a-addr - x )` and `C@ ( c-addr - char )` meet at `( a-addr - x )`, because `x`, not `char`, is the postcondition guaranteed by both. For a complete conditional, the declared `IF ( flag - )` effect is composed with that branch contract, so the selector is consumed as well.

The operation should not be read as a union of all concrete executions. It is a *must* contract: one input/output description valid for both alternatives. Contravariant inputs make every arm executable; covariant outputs avoid promising a result type or value identity that only one arm provides. It catches the common Forth error where the paths have different net stack changes while allowing different visible depths that describe the same implicit frame.

Control syntax is not hard-wired into the effect algebra. A profile can describe a source form and how its captured regions are reduced:

```
syntax:
  IF <then branch> [ELSE <else branch>] FI
effect:
  IF
  either <then branch> <else branch>
```

The small template language sequences control-word effects and captured regions, combines regions with `either`, and applies `repeat` to a loop region. Consequently systems using `THEN`, `FI`, or a custom immediate word can share the evaluator by changing profile data.

# 5 What the loop check does—and does not do

For a repeated region with effect $p$, the implementation computes one copy $p$, composes two copies $p \cdot p$, and asks for their common branch-style contract:

$$p_2^* = p \sqcap (p \cdot p).$$

This is a finite stack-stability test. It is useful for conventional loops whose iteration restores the same visible stack shape, and it rejects many bodies that grow, shrink, or change a cell incompatibly on the second iteration.

It is not Kleene star, transitive closure, or a general abstract-interpretation fixpoint. Agreement of one and two iterations alone does not prove agreement for every iteration count. Nor does the checker discover a programmer's loop invariant. Profiles for `BEGIN ... UNTIL` or `BEGIN ... WHILE ... REPEAT` must include the test word in the repeated region when its per-iteration `flag` consumption belongs there. Calling this rule "idempotent repetition" is convenient historically, but "one-versus-two approximation" describes the implemented guarantee more accurately.

For production assurance, the replacement is standard: construct a control-flow graph, maintain an abstract stack at each loop header, and iterate to a post-fixpoint, widening if the chosen domain requires it. The local rule remains attractive for a small Forth tool because it terminates immediately and works without constructing a whole-program graph.

# 6 Following the outer interpreter

A Forth source checker cannot tokenize every blank-delimited string as an ordinary word. The specification file therefore includes source behavior as well as runtime stack effects:

```
"("       parse until ")"          ( -- )
"\\"      parse until eol          ( -- )
CONSTANT  parse word define        ( x -- )
```

```
VARIABLE  parse word define        ( -- a-addr )
:         parse word define colon  ( -- )
;         control end              ( -- )
literal integer                    ( -- n )
IF        control if               ( flag -- )
```

The evaluator has interpretation and compilation states. Defining words consume the following name and extend the analysis dictionary; colon bodies are checked and their inferred effects are installed at `;`. Parser words consume names or delimited text. Literals are classified by profile entries. Word lookup is case-insensitive.

An ordinary comment such as `( n - n )` is consumed, not trusted as an assertion. Otherwise a false source comment could override the very inference being checked. Recognized locals syntax may supply a provisional definition shape, but that is a distinct parser feature. Return-stack activity, `EXECUTE`, native code generation, `THROW`, and unrestricted metaprogramming require special abstractions or remain outside this model.

Once one colon body has been parsed, evaluation is simple. Leaves are known word or literal effects; adjacent nodes use composition; control nodes apply the profile template. Dependencies run from a node to its contained source regions, so a non-recursive parsed body has one deterministic bottom-up result. This modest observation is more useful than a heavy theorem here: it identifies exactly what a reproducible diagnostic depends on—the parsed structure, the current dictionary, and the three effect operators.

# 7 Complete profile example

The bundled `real` profile is run with

```
./run-evaluator-gforth.sh real
```

It reads `ex1types.txt`, `ex1specs.txt`, and `ex1prog.txt`. The source defines:

```
: PEEK2   DUP @ SWAP @ ;
: MAYSWAP IF SWAP FI ;
: NLOOP   BEGIN DUP 0= WHILE REPEAT ;
: ULOOP   BEGIN DUP 0= UNTIL ;
: KEEPIF  IF DUP ELSE DUP FI ;

DP DP C@ 0= KEEPIF
```

Representative profile declarations are `@ ( a-addr - x )`, `C@ ( c-addr - char )`, `0= ( n - flag )`, and the indexed stack operators shown earlier. The inferred definitions are:

```
PEEK2   ( a-addr -- x x )
MAYSWAP ( x x flag -- x x )
NLOOP   ( n[1] -- n[1] )
ULOOP   ( n[1] -- n[1] )
KEEPIF  ( x[1] flag -- x[1] x[1] )
```

The top-level phrase ends in

```
DP      ( -- a-addr[1] )
DP      ( -- a-addr )
C@      ( a-addr -- char )
0=      ( char -- flag )
KEEPIF  ( a-addr[1] flag -- a-addr[1] a-addr[1] )
< a-addr[1] a-addr[1]
```

`C@` asks for `c-addr`; `DP` provides the more specific `a-addr`, so the boundary succeeds. Since `char` is below `n`, `0=` also succeeds. Both `KEEPIF` paths duplicate the surviving address and their common contract retains that identity. This one example exercises subtyping, symbolic copies, a definition, branch merge, loop approximation, literals, and top-level checking.

# 8 Implementation and evidence

The native implementation is one Gforth source file. Its word families separate type relations (`ev-ts-*`), symbols (`ev-sym-*`), effects (`ev-spec-*`), effect lists, specification dictionaries, source parsing, and diagnostics. Effects are mutable records, so dictionary entries are deep-cloned before use and independently evaluated branches receive disjoint wildcard ranges. Input line endings and normalized wildcard allocation are made deterministic across runs.

Boundary matching itself is linear in the shorter touching boundary. The current substitution strategy scans whole working effects after each match, so large intermediate effects can make reduction roughly quadratic. A union–find representation of identity classes and persistent stack rows would reduce copying, but the straightforward implementation has proved easier to inspect in Gforth.

The checked-in regression corpus contains 29 programs: 13 expected accepts and 16 expected rejects. A current native batch accepts all 13 positive cases and rejects all 16 negative cases with diagnostics. This is evidence about the artifact, not a soundness result. The profile is also an assumption: tests cannot reveal an error shared by a primitive declaration and its expected output.

# 9 Scope and next steps

The checker preserves the main benefits sought by a Forth programmer: inferred stack comments, early detection of type or depth clashes, dataflow through stack shuffles, address-type refinement, and profile-defined immediate syntax. Its limits should remain visible:

- primitive and parser specifications are trusted;
- only comparable types unify;
- compaction can lose implicit identity information;
- branch merge is a single variance-aware common contract, not a set of path-specific effects;
- loop checking compares only one and two iterations;
- recursion, forward references, quotations, non-local exits, and unrestricted compile-time behavior are not covered generally.

The most direct improvements are to retain raw correlations until display, fix the top-level clash path, and replace repeated global substitution by identity classes. Explicit stack-row variables would support more general stack-polymorphic and higher-order words. A control-flow fixpoint would give loops a conventional invariant semantics. Finally, checking primitive specifications against an operational model is necessary before claiming whole-system soundness.

Forth stack comments already provide the right interface notation. The main step is to make their variables, subtype relationships, parser effects, and control combinations executable. Symbolic stack effects do that with a small set of understandable operations: subtype-aware boundary composition for sequences, common contracts for alternatives, and a clearly delimited local approximation for repetition. The result is not a static semantics for all of Forth, but it is a practical and extensible checker for the substantial subset that a system profile can describe.

# References


[1] Forth 200x Standardisation Committee. *Forth 2012 Standard*, 2014. `https://forth-standard.org/standard/intro`.

[2] J. Pöial. Algebraic specifications of stack effects for Forth programs. In *1990 FORML Conference Proceedings*, pp. 282–290, 1991.

[3] J. Pöial. Algebraic specifications of stack effects. *Forth Dimensions*, 16(4):18–20, 1994.

[4] J. Pöial. Stack effect calculus with typed wildcards, polymorphism and inheritance. *18th EuroForth Conference*, 2002.

[5] J. Pöial. Typing tools for typeless stack languages. In *22nd EuroForth Conference*, pp. 40–46, 2006.

[6] J. Pöial. Java framework for static analysis of Forth programs. In *24th EuroForth Conference*, pp. 20–23, 2008.

[7] B. Stoddart and P. J. Knaggs. Type inference in stack based languages. *Formal Aspects of Computing*, 5(4):289–298, 1993.